UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

CAHIER D'ANALYSE

SÉCURITÉ DU CLOUD

THÈME : Stockage objet évolutif avec Swift

Réalisé par :

- FOUDA DZOU AMBE Valdez
- KAMDOUM TEUFENG Josette

Sous la coordination de : Mr. NGUIMBUS Emmanuel

Année Académique
2025-2026

INTRODUCTION **3**
I- PRÉSENTATION 4
A. CONTEXTE 4
B. DEFINITION DU PROBLEME 4
II- BESOIN DU SYSTÈME 4
A. BESOINS FONCTIONNELS 4
B. BESOINS NON FONCTIONNELS 5
III- CAS D'UTILISATION DU SYSTÈME 5
A. ACTEURS INTERNES 5
B. ACTEURS EXTERNES 6
C. CAS D'UTILISATIONS PROPREMENT DIT 6
D. DIAGRAMME DE CAS D'UTILISATION 6
CONCLUSION **7**

INTRODUCTION

Le stockage objet évolutif est un enjeu majeur dans la gestion moderne des données, notamment dans les environnements cloud. Ce projet vise à développer une solution de stockage objet basée sur OpenStack Swift, capable de fonctionner de manière similaire à OneDrive, en offrant un service de synchronisation et d'accès fluide aux données pour les utilisateurs. L'objectif est d'assurer un stockage distribué, sécurisé et hautement disponible, avec une gestion intelligente des fichiers permettant la synchronisation automatique entre différents appareils et la sauvegarde des données dans le cloud. Ce cahier d'analyse présente les exigences fonctionnelles et techniques indispensables, les contraintes liées à la scalabilité et à la performance, ainsi que les choix d'architecture justifiant l'adoption de Swift pour répondre aux besoins spécifiques d'un stockage objet évolutif à l'image des solutions comme OneDrive.

I- PRÉSENTATION

A. CONTEXTE

Aujourd'hui, la gestion des données numériques connaît une croissance rapide, nécessitant des solutions de stockage capables d'évoluer facilement et de manière fiable. Le stockage objet, grâce à des systèmes distribués comme OpenStack Swift, permet de stocker et d'organiser efficacement de très grands volumes de données en les répartissant sur plusieurs serveurs. Cette méthode offre une grande flexibilité, une haute disponibilité et une sécurité renforcée.Notre projet s'inscrit donc dans ce contexte en visant à développer une solution de stockage objet évolutive, performante et adaptée aux besoins actuels des utilisateurs fonctionnant comme Onedrive.

B. DEFINITION DU PROBLEME

L'explosion des données numériques nécessite des solutions de stockage scalables et fiables, particulièrement sous Linux où les alternatives à OneDrive font défaut. OpenStack Swift excelle en scalabilité horizontale et haute disponibilité via stockage objet distribué, mais son interface technique (API REST, CLI) reste inaccessible aux utilisateurs non-experts, contrairement à l'expérience fluide d'OneDrive avec synchronisation automatique, partage simplifié et recherche visuelle.Swift gère efficacement les volumes massifs de données mais manque d'une vue d'ensemble intuitive pour organiser des fichiers sous Linux, tandis qu'il n'offre pas de client natif assurant une synchronisation bidirectionnelle temps réel multi-appareils. Sa résilience native aux pannes via réplication nécessite une configuration complexe sans interface graphique accessible, et son interface web n'est pas ergonomique. Ces lacunes créent un verrou propriétaire, forçant les utilisateurs Linux à dépendre de solutions Windows.

II- BESOIN DU SYSTÈME

A. BESOINS FONCTIONNELS

Les besoins fonctionnels de notre système de stockage visent à reproduire et surpasser l'expérience OneDrive sous Linux, en assurant une gestion intuitive des données massives, une synchronisation transparente multi-appareils, une haute disponibilité avec tolérance aux pannes, et une interface utilisateur fluide malgré la complexité technique sous-jacente. Notre système aura donc pour objectif d'assurer :

- Un stockage et gestion de données non structurées
  Le système doit pouvoir stocker et organiser efficacement de grandes quantités de données non structurées (images, vidéos, documents, etc.) sous forme d'objets avec métadonnées associées. (géré par swift)
- Une évolutivité horizontale
  La solution doit permettre d'ajouter facilement des nœuds de stockage pour augmenter

la capacité sans interruption de service, garantissant une croissance continue et transparente.(géré par swift)

- Une Haute disponibilité et tolérance aux pannes
  Les données doivent être répliquées automatiquement sur plusieurs nœuds ou centres de données, assurant ainsi une continuité de service même en cas de défaillance matérielle.(géré par swift)
- Une Synchronisation et cohérence des données
  Le système doit permettre la synchronisation automatique des données locales entre différents appareils et assurer la cohérence des fichiers malgré les modifications simultanées réalisées en local.
- Versioning des objets

B. BESOINS NON FONCTIONNELS

Les besoins non fonctionnels de notre système de stockage visent à garantir :

- Scalabilité
  La capacité de stockage et la performance doivent pouvoir croître de manière linéaire avec l'ajout de nouvelles ressources sans perte de qualité de service.
- Disponibilité élevée
  Le service doit rester accessible en permanence, avec une tolérance aux pannes grâce à la redondance et la réplication des données.
- Sécurité des données
  Il est impératif d'assurer la confidentialité, l'intégrité et la protection des données, notamment via le chiffrement, les contrôles d'accès et la gestion des identités.
- Fiabilité et résilience
  Le système doit être capable de résister aux défaillances matérielles ou logicielles sans perte de données ni interruption durable de service.
- Interopérabilité
  La solution doit pouvoir s'intégrer facilement avec les systèmes existants via des API standard (RESTful) et supporter différents protocoles de communication.

Expérience utilisateur fluide
Malgré la complexité du système en arrière-plan, l'accès et l'utilisation des données doivent être simples, intuitifs et transparents pour les utilisateurs finaux.

III- CAS D'UTILISATION DU SYSTÈME

A. ACTEURS INTERNES

ACTEURS	RÔLES
Utilisateurs	• Installer le client Swift sur sa machine • Restaurer une version ultérieure du stockage

	• Activer le versioning • Modifier les fichiers • se connecter sur la plateforme • se déconnecter sur la plateforme • création du fichier
Client système	• Mettre à jour les fichiers • suppression de fichiers • Authentification au serveur OpenStack

B. ACTEURS EXTERNES

ACTEURS	RÔLES
Plateforme	• Appels d'API vers openstack swift • Visualisation des différentes versions
OpenStack Swift	• Réplication des données sur plusieurs sites • Versioning des données • Sauvegarde des données

C. CAS D'UTILISATIONS PROPREMENT DIT

Titre :	Installer le Client Swift	
Objectif :	Permettre à l'utilisateur d'installer et configurer le client Swift pour la synchronisation automatique des fichiers entre sa machine locale et la plateforme cloud.	
Résumé :	L'utilisateur télécharge et installe le client Swift sur son ordinateur, permettant ainsi la synchronisation automatique des fichiers sélectionnés avec la plateforme de stockage cloud	
Acteur :	Utilisateur, Client Swift, Plateforme	
Présupposé	**Préconditions**	**Post conditions**
• *L'utilisateur dispose d'une machine compatible avec le client Swift* • *Le système d'exploitation est supporté (Linux)* • *L'utilisateur a les droits administrateur nécessaires pour l'installation*	• *L'utilisateur dispose d'un compte valide sur la plateforme* • *L'utilisateur a les droits d'administration sur sa machine* • *La connexion internet est disponible*	• *Le client Swift est installé et configuré* • *L'utilisateur peut sélectionner les dossiers à synchroniser*
Scénario nominal :		
1. L'utilisateur se rend sur le site web de la plateforme 2. L'utilisateur télécharge le client Swift pour son système d'exploitation 3. L'utilisateur lance l'installateur 4. L'installateur copie les fichiers nécessaires 5. L'utilisateur configure le dossier de synchronisation par défaut 6. Le client se connecte à la plateforme pour vérification 7. L'installation se termine avec succès 8. Le client démarre la synchronisation initiale		
Scénario alternatif :		
1. Le téléchargement échoue en raison d'une connexion instable 2. Le système propose de reprendre le téléchargement 3. L'utilisateur accepte et le téléchargement reprend 4. Retour au scénario nominal à l'étape 3		

<table>
<tr><td>Titre :</td><td colspan="2">Restaurer une version antérieure d’un fichier</td></tr>
<tr><td>Objectif :</td><td colspan="2">Permettre la récupération d’un fichier sauvegardé à partir de Swift, y compris une version antérieure</td></tr>
<tr><td>Résumé :</td><td colspan="2">L’utilisateur sélectionne un fichier et une version spécifique à restaurer depuis Swift vers l’environnement local.</td></tr>
<tr><td>Acteur :</td><td colspan="2">utilisateur, serveur swift</td></tr>
<tr><td>Présupposé</td><td>Préconditions</td><td>Post conditions</td></tr>
<tr><td><ul><li>Un fichier a été supprimé ou corrompu localement</li></ul></td><td><ul><li>Le fichier existe dans Swift</li><li>L’utilisateur est autorisé à restaurer les données</li></ul></td><td><ul><li>Le fichier est restauré localement</li></ul></td></tr>
<tr><td colspan="3">Scénario nominal :</td></tr>
<tr><td colspan="3"><ol><li>L’utilisateur sélectionne le fichier à restaurer.</li><li>Il choisit la version souhaitée.</li><li>L’application envoie la requête à Swift.</li><li>Swift transmet le fichier.</li><li>Le fichier est restauré localement</li></ol></td></tr>
<tr><td colspan="3">Scénario alternatif :</td></tr>
<tr><td colspan="3"><ol><li>Version inexistante → message d’erreur</li><li>Connexion interrompue → restauration partielle annulée</li></ol></td></tr>
</table>

Titre :	Modifier les fichiers	
Objectif :	Permettre à l'utilisateur de modifier un fichier existant dans le stockage cloud.	
Résumé :	L'utilisateur ouvre, modifie et sauvegarde un fichier stocké sur la plateforme Swift.	
Acteur :	utilisateur, client swift, serveur swift	
Présupposé	**Préconditions**	**Post conditions**
● *L'utilisateur dispose des applications nécessaires pour modifier le type de fichier* ● *Le fichier n'est pas verrouillé par un autre utilisateur ou processus*	● *Le fichier existe dans le stockage* ● *L'utilisateur a les droits d'écriture* ● *L'application d'édition appropriée est disponible*	● *Le fichier est modifié* ● *Une nouvelle version est créée (si versioning activé)* ● *La synchronisation est déclenchée*
Scénario nominal :		
1. L'utilisateur sélectionne un fichier 2. L'utilisateur clique sur "Modifier" 3. Le fichier est téléchargé localement 4. L'application d'édition appropriée s'ouvre 5. L'utilisateur effectue les modifications 6. L'utilisateur sauvegarde le fichier 7. Le fichier est uploadé vers Swift 8. Une nouvelle version est créée 9. La synchronisation est déclenchée 10. Le système confirme la modification		
Scénario alternatif :		
1. Un autre utilisateur a modifié le fichier simultanément 2. Le système détecte le conflit 3. Le système propose de fusionner ou de conserver une version 4. L'utilisateur choisit l'option de résolution 5. Le conflit est résolu 6. Retour au scénario nominal à l'étape 8		

<table>
<tr><td>Titre :</td><td colspan="2">Authentification</td></tr>
<tr><td>Objectif :</td><td colspan="2">Permettre au client Swift de s’authentifier afin d’accéder aux services de stockage objet</td></tr>
<tr><td>Résumé :</td><td colspan="2">Le client Swift s’authentifie auprès de la plateforme OpenStack afin d’obtenir un jeton d’accès lui permettant d’effectuer des opérations sur les objets</td></tr>
<tr><td>Acteur :</td><td colspan="2">client swift, plateforme openstack</td></tr>
<tr><td>Présupposé</td><td>Préconditions</td><td>Post conditions</td></tr>
<tr><td><ul><li>Le client souhaite accéder aux fonctionnalités de stockage Swift</li></ul></td><td><ul><li>Le client possède des identifiants valides</li><li>Les services d’authentification sont disponibles</li></ul></td><td><ul><li>Un jeton d’authentification est délivré</li><li>Le client peut accéder aux API Swif</li></ul></td></tr>
<tr><td colspan="3">Scénario nominal :</td></tr>
<tr><td colspan="3"><ol><li>Le client fournit ses identifiants.</li><li>La plateforme vérifie les informations.</li><li>Un jeton d’authentification est généré.</li><li>Le jeton est transmis au client Swift</li></ol></td></tr>
<tr><td colspan="3">Scénario alternatif :</td></tr>
<tr><td colspan="3"><ol><li>Identifiants invalides → accès refusé</li><li>Service d’authentification indisponible → échec de la connexion</li></ol></td></tr>
</table>

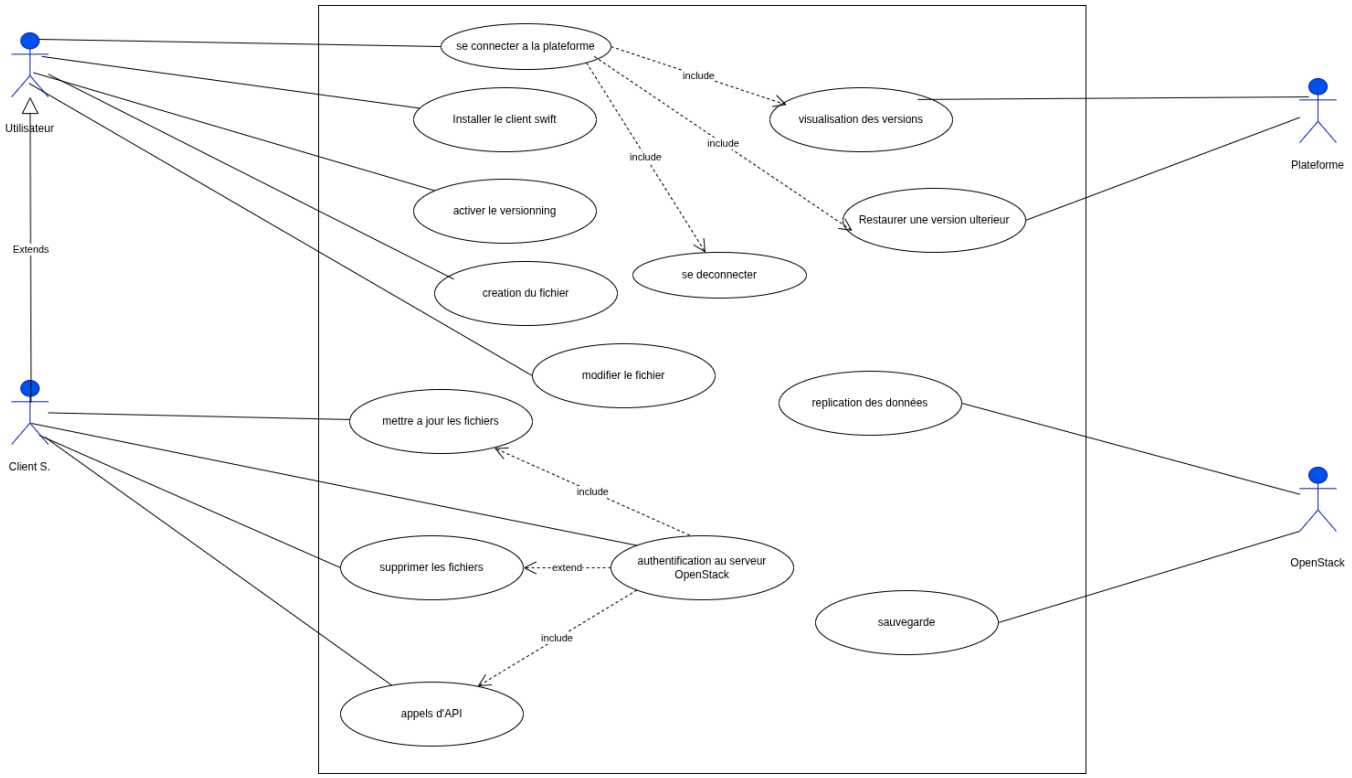
Titre :	Mettre à jour les fichiers	
Objectif :	Permettre au client Swift de modifier un fichier déjà stocké	
Résumé :	Le client met à jour un fichier en envoyant une nouvelle version vers Swift.	
Acteur :	client swift, serveur swift	
Présupposé	**Préconditions**	**Post conditions**
• *Le fichier existe déjà dans le stockage Swift.*	• *Le client est authentifié* • *Le fichier existe dans Swift*	• *Une nouvelle version du fichier est stockée* • *L'ancienne version est conservée*
Scénario nominal :		
1. Le client sélectionne le fichier à modifier. 2. Le client envoie la nouvelle version. 3. La plateforme transmet la requête à Swift. 4. Swift stocke la nouvelle version du fichier		
Scénario alternatif :		
1. Fichier inexistant → erreur retournée 2. Problème réseau → mise à jour annulée		

Titre :	Suppression de fichiers	
Objectif :	Permettre au client Swift de supprimer un fichier stocké	
Résumé :	Le client demande la suppression d'un fichier depuis Swift	
Acteur :		
Présupposé	**Préconditions**	**Post conditions**
• *Le fichier n'est plus nécessaire*	• *Le client est authentifié* • *Le fichier existe dans Swift* •	• *Le fichier est supprimé ou marqué comme supprimé* • *L'opération est journalisée*
Scénario nominal :		
1. Le client sélectionne le fichier. 2. Le client envoie la requête de suppression. 3. Swift supprime l'objet. 4. Une confirmation est envoyée au client		
Scénario alternatif :		
1. Fichier inexistant → message d'erreur 2. Suppression non autorisée → accès refus		

<table>
<tr><td>Titre :</td><td colspan="2">Visualisation des versions d’un fichier</td></tr>
<tr><td>Objectif :</td><td colspan="2">Permettre à l’utilisateur de consulter les différentes versions d’un fichier.</td></tr>
<tr><td>Résumé :</td><td colspan="2">La plateforme interroge Swift pour afficher l’historique des versions disponibles</td></tr>
<tr><td>Acteur :</td><td colspan="2">Plateforme, serveur Swift</td></tr>
<tr><td>Présupposé</td><td>Préconditions</td><td>Post conditions</td></tr>
<tr><td><ul><li>Le fichier possède plusieurs versions</li></ul></td><td><ul><li>Le versioning est activé</li><li>Le fichier existe dans Swift</li></ul></td><td><ul><li>La liste des versions est affichée</li></ul></td></tr>
<tr><td colspan="3">Scénario nominal :</td></tr>
<tr><td colspan="3"><ol><li>La plateforme demande la liste des versions.</li><li>Swift retourne les métadonnées.</li><li>La plateforme affiche les versions disponibles.</li></ol></td></tr>
<tr><td colspan="3">Scénario alternatif :</td></tr>
<tr><td colspan="3"><ol><li>Aucune version disponible → message informatif</li><li>Erreur Swift → affichage impossible</li></ol></td></tr>
</table>

Titre :	Sauvegarde des données	
Objectif :	Stocker durablement les données envoyées par le client	
Résumé :	Swift enregistre les objets et leurs métadonnées sur les nœuds de stockage	
Acteur :	serveur swift	
Présupposé	**Préconditions**	**Post conditions**
● *Des données sont envoyées vers Swift*	● *Swift est opérationnel* ● *Les nœuds de stockage sont disponibles*	● *Les données sont stockées* ● *Les métadonnées sont enregistrées*
Scénario nominal :		
1. Swift reçoit les données. 2. Les objets sont distribués sur les nœuds. 3. La sauvegarde est confirmée		
Scénario alternatif :		
1. Disque plein → sauvegarde échouée 2. Nœud indisponible → redirection automatique		

D. DIAGRAMME DE CAS D'UTILISATION



CONCLUSION

En conclusion, ce cahier d'analyse met en lumière les enjeux majeurs liés au stockage objet évolutif, notamment la gestion efficace de volumes croissants de données non structurées dans un environnement cloud distribué. L'adoption de Swift comme solution permet de répondre aux besoins fonctionnels et non fonctionnels identifiés, en offrant une scalabilité horizontale, une haute disponibilité, et une sécurisation robuste des données. Par ailleurs, son architecture distribuée assure une tolérance aux pannes et une flexibilité indispensables pour s'adapter aux exigences actuelles des utilisateurs, à l'image des services modernes tels que OneDrive. Ainsi, cette analyse prépare le terrain pour une implémentation optimale, garantissant performance, fiabilité et simplicité d'utilisation dans la gestion des données à l'échelle.